

Spring Cloud Gateway — Phase 1: Core Gateway

Goal: Build an API Gateway for your local Doctor, Patient, and User Spring Boot services and understand the complete request flow.

Topics: How Gateway works • Routes • Predicates • Filters • Global Filters • Filter ordering • Custom Filters

1. How Gateway Works

An API Gateway is the single entry point between clients and backend microservices. The client sends requests to the Gateway instead of knowing every service address. The Gateway decides where the request should go and can process the request/response.

Client → API Gateway :8080 → Doctor :8081 / Patient :8082 / User :8083

Example: GET /doctors/123 enters the Gateway. The Gateway finds a matching route, evaluates its predicates, runs applicable filters, forwards the request to Doctor Service, receives the response, performs response-side processing, and returns the response to the client.

Remember: Gateway is the traffic entry point and request-processing layer; it is not your business service.

2. Routes

A Route tells Spring Cloud Gateway where a request should go. A route contains an ID, destination URI, predicates, and optionally filters.

```
spring:
  cloud:
    gateway:
      routes:
        - id: doctor-service
          uri: http://localhost:8081
          predicates:
            - Path=/doctors/**
        - id: patient-service
          uri: http://localhost:8082
          predicates:
            - Path=/patients/**
        - id: user-service
          uri: http://localhost:8083
          predicates:
            - Path=/users/**
```

Think: **Route** = “If this request matches these conditions, send it to this destination and apply these filters.”

3. Predicates

A Predicate is a condition used to determine whether a route matches an incoming request.

For Phase 1, focus on Path, Method, Header, and Query predicates.

```
GET /doctors/123 + Path=/doctors/** → MATCH
GET /patients/123 + Path=/doctors/** → NO MATCH
```

Path checks the URL path. Method checks GET/POST/etc. Header checks a request header. Query checks query parameters.

4. Filters — Detailed

A Gateway Filter processes a request or response as it passes through Gateway. Filters can add/remove headers, rewrite paths, modify request/response information, log data, or implement Gateway behavior.

```
Client Request
↓
Filter processing
↓
Downstream Service
↓
Response processing
↓
```

Client

4.1 Route Filter vs Global Filter

A route filter is attached to a particular route. A `GlobalFilter` participates across Gateway routes. Use a route filter for route-specific behavior and a `GlobalFilter` for common cross-route behavior.

4.2 Built-in Filters to Practice

AddRequestHeader: adds a request header before forwarding. Example: `AddRequestHeader=X-Gateway,true`.

AddResponseHeader: adds a header to the response returned by Gateway.

RemoveRequestHeader: removes a request header before the downstream call.

RemoveResponseHeader: removes a response header.

StripPrefix: removes path segments before forwarding. Test the exact path received by the service.

RewritePath: transforms the path using a pattern, giving more control than simply removing a fixed number of segments.

SetPath: sets the downstream request path to a chosen value.

PrefixPath: adds a prefix to the request path.

4.3 Example

```
spring:
  cloud:
    gateway:
      routes:
        - id: doctor-service
          uri: http://localhost:8081
          predicates:
            - Path=/doctors/**
          filters:
            - AddRequestHeader=X-Gateway,true
```

Call `/doctors` through Gateway and verify Doctor Service receives `X-Gateway=true`. This proves the filter ran before forwarding.

4.4 Pre-processing and Post-processing

Conceptually, filters can do work before the downstream call and after the downstream response. For example, add a request header before forwarding and log/inspect the response after the service responds.

Request → pre-processing → Service → post-processing → Response

4.5 Why Filters Matter

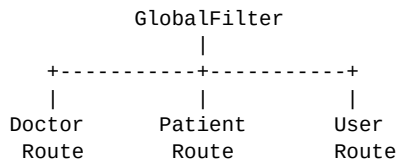
Filters centralize cross-cutting Gateway behavior instead of duplicating it in every microservice. Later phases will use filters for authentication, rate limiting, retries, circuit breakers, tracing, and other concerns.

4.6 Filter Chain

```
Incoming request
↓
Global / route filters
↓
Request processing
↓
Downstream service
↓
Response processing
↓
Client
```

5. Global Filters

A `GlobalFilter` can participate across Gateway routes. Typical uses include request logging, correlation IDs, common headers, and other cross-cutting processing.



Exercise: create a GlobalFilter that logs HTTP method and request path. Call /doctors, /patients, and /users and verify it runs for all.

6. Filter Ordering

Multiple filters can participate in one request. Their order matters because one filter may depend on work done by another.

Request → Filter A → Filter B → Filter C → Service

Create two simple filters that log their execution. Change their order and observe the difference. Understand that pre/post behavior can make the observed logs look different from a simple top-to-bottom list.

7. Custom Filters

Create a custom filter when the built-in Gateway filters do not implement the behavior you need. Prefer built-in filters when they already solve the problem.

Recommended exercise: require X-Client-Type for Doctor Service.

```

Request
↓
Custom Filter
↓
X-Client-Type exists?
  ■■ No → reject
  ■■ Yes → continue → Doctor Service
  
```

Test without the header and confirm rejection. Then send X-Client-Type: web and confirm the request reaches Doctor Service. Understand the distinction between a custom route GatewayFilter and a custom GlobalFilter.

8. Your Practical Phase 1 Build Plan

Step 1. Verify Doctor, Patient, and User services work directly.

Step 2. Create the Gateway and run it on port 8080.

Step 3. Create /doctors/** → Doctor, /patients/** → Patient, /users/** → User routes.

Step 4. Test all three routes through Gateway.

Step 5. Experiment with Path, Method, Header, and Query predicates.

Step 6. Practice AddRequestHeader, AddResponseHeader, RemoveRequestHeader, StripPrefix, RewritePath, SetPath, and PrefixPath.

Step 7. Create a GlobalFilter that logs method and path.

Step 8. Create two filters and practice filter ordering.

Step 9. Create a custom X-Client-Type validation filter for Doctor Service.

Step 10. Run a final end-to-end test and explain every stage.

9. Final Architecture

```

Client / Postman
↓
API Gateway :8080
  ■■ Routes
  ■■ Predicates
  ■■ Filters
  ■■ GlobalFilter
  ■■ Filter Ordering
  ■■ Custom Filter
  
```

↓

Doctor :8081 Patient :8082 User :8083

10. Phase 1 Completion Checklist

- Explain why an API Gateway is placed in front of microservices.
- Explain the complete request/response flow.
- Create routes for Doctor, Patient, and User services.
- Explain and practice Path, Method, Header, and Query predicates.
- Explain and use built-in filters.
- Explain route filters vs GlobalFilters.
- Create and test a GlobalFilter.
- Explain why filter ordering matters.
- Create and test a custom filter.
- Troubleshoot a request that does not reach the expected service.

11. Mental Model to Remember

Client → Gateway → find matching Route → evaluate Predicates → execute Filter Chain → Microservice → response processing → Client

Predicate: Should this route handle the request? **Route:** Where should it go? **Filter:** What processing should happen?
GlobalFilter: What processing is common across routes?